

Automação de Testes

O que automatizar, arquitetura, Playwright, Selenium, Robot Framework e manutenção sustentável.

O que é automação de testes?

Automação de testes usa código e ferramentas para executar verificações de forma repetível, comparar resultados e produzir evidências. Ela não substitui pensamento crítico, exploração ou análise de risco; automatiza verificações que fazem sentido repetir.

Para que serve

Reduz trabalho repetitivo, acelera feedback, amplia frequência de regressão, apoia CI/CD e libera tempo humano para investigação e testes que exigem julgamento. O valor vem de feedback confiável, não da quantidade de scripts.

Quando automatizar

Bons candidatos costumam ser cenários estáveis, repetitivos, críticos, executados com frequência e com resultado objetivo. Considere custo de manutenção, dados, ambiente, dependência externa e velocidade de feedback.

Quando não automatizar primeiro

Funcionalidade instável, protótipo descartável, fluxo executado raramente, validação altamente subjetiva ou cenário cujo custo de automação supera o benefício podem ser prioridades menores. A decisão deve ser baseada em risco e retorno.

Pirâmide e camadas

Uma estratégia saudável combina verificações em diferentes níveis. Testes menores tendem a ser mais rápidos e baratos; E2E cobrem fluxos amplos, mas costumam ser mais lentos e frágeis. O equilíbrio depende da arquitetura e do risco.

Playwright, Selenium, Cypress e Robot Framework

São opções com características e ecossistemas diferentes. Playwright oferece automação moderna de browsers e APIs; Selenium possui amplo ecossistema WebDriver; Cypress tem arquitetura e experiência próprias; Robot Framework usa abordagem keyword-driven e bibliotecas extensivas.

Arquitetura de automação

Separe dados, configuração, locators e regras reutilizáveis. Fixtures, Page Objects quando adequados, helpers, configuração por ambiente e convenções de nomes reduzem duplicação e tornam a suite mais fácil de evoluir.

Flaky tests

Um flaky test passa e falha sem mudança relevante no produto. Causas incluem sincronização, dados compartilhados, ambiente instável, dependências externas e selectors frágeis. Retry pode reduzir ruído temporariamente, mas não deve esconder a causa.

Evidências e debugging

Screenshots, vídeos, traces, logs, console e network ajudam a investigar falhas. A automação deve produzir informação suficiente para responder onde, quando e por que a verificação falhou.

Automação em CI/CD

No pipeline, testes podem rodar em pull requests, merges, builds ou schedules. Suites devem ser organizadas por velocidade e risco: checks rápidos cedo; suites mais pesadas em etapas adequadas; resultados e artefatos precisam ficar acessíveis ao time.